

TD4 :

Orinté objet

Compte rendu du TD4

Noms : MAALEJ Mariem

Date : 20/02/2026

Table des matières

1	La classe Main :	2
1.1	Usage de Swing :	2
1.2	Interface Engine :	2
1.2.1	Squelette de la classe RenderEngine :	3
2	RenderEngine :	4
2.1	La classe Sprite :	5
2.1.1	Interface Displayable :	5
2.1.2	Classe Sprite :	6
2.2	La classe RenderEngine :	7
2.2.1	Surcharge de la méthode <code>paint(Graphics g)</code> :	8
2.2.2	Code de la classe RenderEngine :	8
2.2.3	Test dans la classe Main :	9

1 La classe Main :

1.1 Usage de Swing :

Le constructeur de la classe `Main` permet d'afficher une fenêtre graphique de taille 400×600 pixels en utilisant le framework `Swing`.

```
import javax.swing.JFrame;

public class Main {

    JFrame displayZoneFrame;

    public Main() throws Exception {
        displayZoneFrame = new JFrame("Java Labs");
        displayZoneFrame.setSize(400,600);
        displayZoneFrame.setDefaultCloseOperation(JFrame.
            EXIT_ON_CLOSE);
        displayZoneFrame.setVisible(true);
    }

    public static void main(String[] args) throws Exception {
        Main main = new Main();
    }
}
```

Listing : La base de notre projet

Explication du code : Ce code permet de créer la fenêtre principale de l'application en utilisant `Swing`.

- La variable `displayZoneFrame` est un objet de type `JFrame` qui représente cette fenêtre.
- Dans le constructeur de la classe `Main`, une fenêtre intitulée `Java Labs` est créée, puis sa taille est fixée à 400×600 pixels.
- La méthode `setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)` permet de fermer complètement le programme lorsque l'utilisateur clique sur la croix de la fenêtre.
- La méthode `setVisible(true)` affiche la fenêtre à l'écran.
- La méthode `main` lance le programme en créant un objet `Main`.

1.2 Interface Engine :

Dans cette partie, on doit créer une interface appelée `Engine` contenant une seule méthode nommée `update()` qui ne retourne rien.

Code:

```
public interface Engine {
    void update();
}
```

Explication : Une interface en Java permet de définir un ensemble de méthodes sans donner leur implémentation. Toute classe qui implémente cette interface devra obligatoirement définir la méthode `update()`.

1.2.1 Squelette de la classe `RenderEngine` :

Dans cette partie, on doit créer la classe `RenderEngine` en lui faisant implémenter l'interface `Engine`. La méthode `update()` doit être définie et, pour vérifier son bon fonctionnement, on affiche temporairement un message dans la console à chaque appel.

Code de la classe `RenderEngine`:

```
public class RenderEngine implements Engine {

    public RenderEngine() {
    }

    @Override
    public void update() {
        System.out.println("Render update");
    }
}
```

La classe `RenderEngine` implémente l'interface `Engine`, elle doit donc définir obligatoirement la méthode `update()`. Ici, cette méthode affiche simplement le message `Render update` dans la console, ce qui permet de vérifier qu'elle est bien appelée régulièrement.

Code de la classe `Main`:

```
import javax.swing.JFrame;
import javax.swing.Timer;
public class Main {
    JFrame displayZoneFrame;
    RenderEngine renderEngine;

    public Main() throws Exception {
        displayZoneFrame = new JFrame("Java Labs");
        displayZoneFrame.setSize(400,600);
        displayZoneFrame.setDefaultCloseOperation(JFrame.
            EXIT_ON_CLOSE);

        renderEngine = new RenderEngine();

        Timer renderTimer = new Timer(50, (time) -> renderEngine.
            update());

        renderTimer.start();
        displayZoneFrame.setVisible(true);
    }

    public static void main(String[] args) throws Exception {
        Main main = new Main();
    }
}
```

Explication : Dans la classe `Main`, on commence par créer une fenêtre `JFrame` de taille 400×600 pixels. Ensuite, on instancie la classe `RenderEngine`.

La ligne :

```
Timer renderTimer = new Timer(50, (time) -> renderEngine.update());
```

crée un timer qui appelle la méthode `update()` toutes les 50 millisecondes. Ainsi, le moteur de rendu est exécuté de manière périodique.

Enfin, `renderTimer.start()` démarre le timer et `displayZoneFrame.setVisible(true)` affiche la fenêtre.

Cette étape permet de mettre en place le squelette du moteur de rendu. Même si aucun affichage graphique réel n'est encore présent, on vérifie déjà que la méthode `update()` est appelée automatiquement et régulièrement.

2 RenderEngine :

Le framework Swing affiche les éléments graphiques dans un panneau. Pour que l'affichage soit correctement synchronisé, la classe `RenderEngine` doit hériter de la classe `JPanel`.

Modification de la classe `RenderEngine`:

```
import javax.swing.JPanel;

public class RenderEngine extends JPanel implements Engine {

    public RenderEngine() {
        super();
    }

    @Override
    public void update() {
        System.out.println("Render update");
    }
}
```

La classe `RenderEngine` hérite maintenant de la classe `JPanel`, ce qui permet d'utiliser ce panneau pour afficher des éléments graphiques dans la fenêtre.

Rôle de `super()` : Dans le constructeur de la classe `RenderEngine`, on utilise :

```
super();
```

Le mot-clé `super` permet d'appeler le constructeur de la classe parente.

Cela permet d'initialiser correctement le panneau graphique avant d'utiliser ses fonctionnalités.

Modification de la classe `Main`:

```
import javax.swing.JFrame;
import javax.swing.Timer;

public class Main {
```

```
JFrame displayZoneFrame;  
RenderEngine renderEngine;  
  
public Main() throws Exception {  
  
    displayZoneFrame = new JFrame("Java Labs");  
    displayZoneFrame.setSize(400,600);  
    displayZoneFrame.setDefaultCloseOperation(JFrame.  
        EXIT_ON_CLOSE);  
  
    renderEngine = new RenderEngine();  
  
    displayZoneFrame.getContentPane().add(renderEngine);  
  
    Timer renderTimer = new Timer(50, (time) -> renderEngine.  
        update());  
  
    renderTimer.start();  
    displayZoneFrame.setVisible(true);  
}  
  
public static void main(String[] args) throws Exception {  
    Main main = new Main();  
}  
}
```

Explication : Dans la classe Main, la ligne

```
displayZoneFrame.getContentPane().add(renderEngine);
```

permet d'ajouter l'objet `RenderEngine` dans la fenêtre. Ainsi, le panneau devient la zone d'affichage principale.

2.1 La classe Sprite :

Cette partie consiste à créer une classe permettant de représenter n'importe quel élément affichable dans la fenêtre graphique.

2.1.1 Interface Displayable :

On commence par créer une interface `Displayable` contenant une seule méthode abstraite permettant de dessiner un objet à l'écran.

```
import java.awt.Graphics;  
  
public interface Displayable {  
  
    void draw(Graphics g);  
  
}
```

2.1.2 Classe Sprite :

La classe `Sprite` implémente l'interface `Displayable`. Elle représente un objet affichable possédant une image, une position et une taille.

```
import java.awt.Graphics;
import java.awt.Image;

public class Sprite implements Displayable {

    protected Image image;
    protected double x;
    protected double y;
    protected double width;
    protected double height;

    public Sprite(Image image, double x, double y,
                  double width, double height) {

        this.image = image;
        this.x = x;
        this.y = y;
        this.width = width;
        this.height = height;
    }

    @Override
    public void draw(Graphics g) {

        g.drawImage(image,
                    (int)x,
                    (int)y,
                    (int)width,
                    (int)height,
                    null);
    }
}
```

Explication : L'interface `Displayable` définit une méthode `draw(Graphics g)` qui permet de dessiner un objet.

La classe `Sprite` implémente cette interface afin de pouvoir être affichée dans la fenêtre graphique.

Chaque sprite possède :

- une image
- une position définie par x et y
- une taille définie par $width$ et $height$

Le constructeur permet d'initialiser ces cinq attributs.

La méthode

```
g.drawImage(...)
```

permet d'afficher l'image à la position (x, y) avec la taille souhaitée.

2.2 La classe `RenderEngine` :

La classe `RenderEngine` est responsable de l'affichage des éléments graphiques. Elle est appelée régulièrement par la méthode `update()`. On ajoute maintenant les attributs et méthodes nécessaires pour gérer les objets à afficher.

Code:

```
import java.util.ArrayList;
import java.util.List;

public class RenderEngine extends JPanel implements Engine {

    private List<Displayable> renderList;

    public RenderEngine() {
        super();
        renderList = new ArrayList<>();
    }

    public void setRenderList(List<Displayable> renderList) {
        this.renderList = renderList;
    }

    public void addToRenderList(Displayable displayable) {
        renderList.add(displayable);
    }

    @Override
    public void update() {
        repaint();
    }
}
```

Explication : L'attribut

```
private List<Displayable> renderList;
```

contient la liste de tous les objets à afficher à l'écran. Chaque élément doit implémenter l'interface `Displayable`.

Le constructeur initialise la liste avec

```
renderList = new ArrayList<>();
```

La méthode

```
setRenderList(...)
```

permet de remplacer toute la liste des objets à afficher.

La méthode

```
addToRenderList(...)
```

permet d'ajouter un élément dans la liste sans supprimer les autres.
Dans la méthode

```
update()
```

on appelle `repaint()` afin de rafraîchir l'affichage du panneau.

2.2.1 Surcharge de la méthode `paint(Graphics g)` :

On met ici en place un principe important de la programmation orientée objet : le polymorphisme. L'idée est que tous les objets contenus dans la `renderList` sont de type `Displayable`, donc chacun possède une méthode `draw(Graphics g)`. Ainsi, peu importe la classe réelle de l'objet (`Sprite`, `SolidSprite` ou `DynamicSprite`), la bonne méthode `draw()` sera automatiquement appelée.

2.2.2 Code de la classe `RenderEngine` :

```
import java.awt.Graphics;
import java.util.ArrayList;
import java.util.List;
import javax.swing.JPanel;
public class RenderEngine extends JPanel implements Engine {
    private List<Displayable> renderList;
    public RenderEngine() {
        super();
        renderList = new ArrayList<>();
    }

    public void setRenderList(List<Displayable> renderList) {
        this.renderList = renderList;
    }

    public void addToRenderList(Displayable displayable) {
        renderList.add(displayable);
    }

    @Override
    public void paint(Graphics g) {
        super.paint(g);

        for (Displayable d: renderList) {
            d.draw(g);
        }
    }

    @Override
    public void update() {
        repaint();
    }
}
```


Explication : La méthode `paint(Graphics g)` est redéfinie dans la classe `RenderEngine` afin de personnaliser l’affichage du panneau.

La ligne

```
super.paint(g);
```

appelle la méthode `paint` de la classe mère `JPanel`. Cela permet de préparer correctement la zone d’affichage avant de dessiner nos propres éléments.

Ensuite, on parcourt tous les objets contenus dans `renderList` :

```
for (Displayable d: renderList)
```

et on appelle pour chacun la méthode :

```
d.draw(g);
```

Cela permet d’afficher chaque sprite dans la fenêtre.

Dans la méthode `update()`, on appelle `repaint()`. Cette méthode demande à Swing de redessiner le panneau, ce qui provoque automatiquement l’exécution de `paint(Graphics g)`.

2.2.3 Test dans la classe Main :

```
Sprite test = new Sprite(ImageIO.read(new File("./img/tree.png"))
    ,200,300,64,64);

renderEngine.addToRenderList(test);
```

Pour vérifier que tout fonctionne, on crée un sprite de test correspondant à l’image `tree.png`, puis on l’ajoute à la `renderList`. Cela nous affiche un arbre dans la fenêtre.